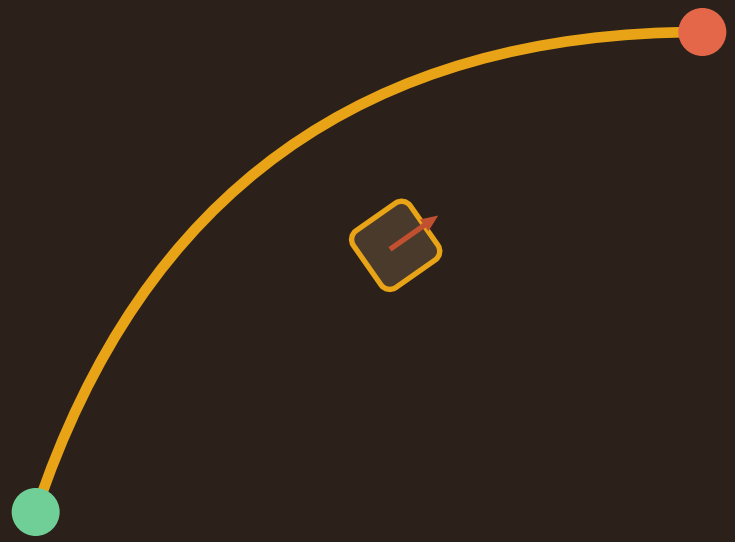
Pedro Pathing in Code

Pedro Pathing 3 for FIRST Tech Challenge

Deck B: from your first path to a full Auto and TeleOp.



Built from pedropathing.com/docs and our PathLab demo.
Deck A explained the ideas; this deck uses them.

THE PLAN

Seven steps, easy to harder.

Step	You will…
1	Draw a path in the Visualizer
2	Write your first Auto: one path
3	Chain many paths with a state machine
4	Choose how the robot faces
5	Flip the Auto for the other alliance
6	Drive in TeleOp, and hand over from Auto
7	Check your work and fix mistakes

Tool	Use it to…
Pedro Visualizer `visualizer.pedropathing.com`	draw paths and see the numbers
FTC Dashboard	see the plan and the robot live; edit PathLab points
Driver Station telemetry	print the pose and the follower mode
Our Tuning Checklist	get the robot ready **before** any of this works well

Read more: Pedro docs: Setting Up Your Autonomous, TeleOp Usage, Debugging; Pedro Visualizer

THE BIG PICTURE

From Constants to a tested robot.

1 Install

Add Pedro to your project (Quickstart or manual).

2 Tune the drivetrain

AutoTune finds your motor names and directions.

`MecanumConfig`

3 Tune the localizer

Pick one (ours: Pinpoint). Set offsets and directions.

`PinpointConfig`

4 Tune Foresight

AutoTune measures top speeds, braking and gains.

`ForesightConfig`

The docs say the order matters: drivetrain, then localizer, then Foresight.

Constants.java holds all three. `Constants.create(hardwareMap)` builds your Follower from them.

5 Test the Follower. Open AutoTune, choose the **Tests** procedure. Seven tests:

Localization Test	drivetrain and localizer together
Driving Test	manual control of the drivetrain
Pose Test	localizer tracks the position
Hold Test	Foresight corrects when you push it
Line Test	straight line, forward and back
Curved Test	a quarter circle, heading follows the path
Interpolation Test	the curve with different heading rules

> This deck starts **after** these tests pass. Our Tuning Checklist is where the team walks through the steps above.

Read more: Pedro docs → Installation, Tuning, Mecanum, Localization, Foresight, Constants, Test

STEP 1 · DRAW IT FIRST

Draw the path before you write the code.

1	Open visualizer.pedropathing.com
2	Put the start point where the robot will sit.
3	Add a line or a curve. Drag the points to shape it.
4	Choose how the robot faces along the path.
5	Read the numbers: x, y and heading of every point.
6	Export the Java and read it. You can now.

Buttons change between versions. The idea does not: **draw, check the numbers, then copy.**



Same field as Deck A

BACK TO THE DEMO

The path drawn on the Dashboard in the demo was a path like this one. Draw the Test 3 curve from slide 4 of Deck A: start (24, 24), control point (60, 24), end (72, 60).

Read more: Pedro Visualizer (visualizer.pedropathing.com); Pedro docs → Creating a Path

Poses first. Then a path between them.

```
// lets us write line(...) and curve(...)
import static com.pedropathing.api.Paths.*;

PoseFactory p = PoseFactory.degrees();

Pose startPose = p.of(24, 24, 0);
Pose park = p.of(48, 48, 90);

Path parkPath() {
    return line(startPose, park)
        .linear(startPose, park);
}
```

Declare Poses and paths at the top of the class, never inside `loop()`.

This line	Means
`p.of(24, 24, 0)`	x, y, heading in degrees
`line(a, b)`	a straight path from a to b
`.linear(a, b)`	heading turns smoothly from a's to b's

Add no heading rule and the robot faces the way it travels.

Read more: Pedro docs → Creating a Pose, Creating a Path

Three methods. Three jobs.

```java
@Override
public void init() {
    follower = Constants.create(hardwareMap);
    follower.setPose(startPose);
}

@Override
public void start() {
    follower.follow(parkPath());
}

@Override
public void loop() {
    follower.update();
}
```

`init()`: build the follower and tell it **where the robot starts**.

`start()`: begin the path. This runs once, when you press PLAY.

`loop()`: **update every pass.** No update, no movement.

The docs start the path with Ivy. We use `follower.follow(path)` and a state machine, the PathLab way.

BACK TO THE DEMO

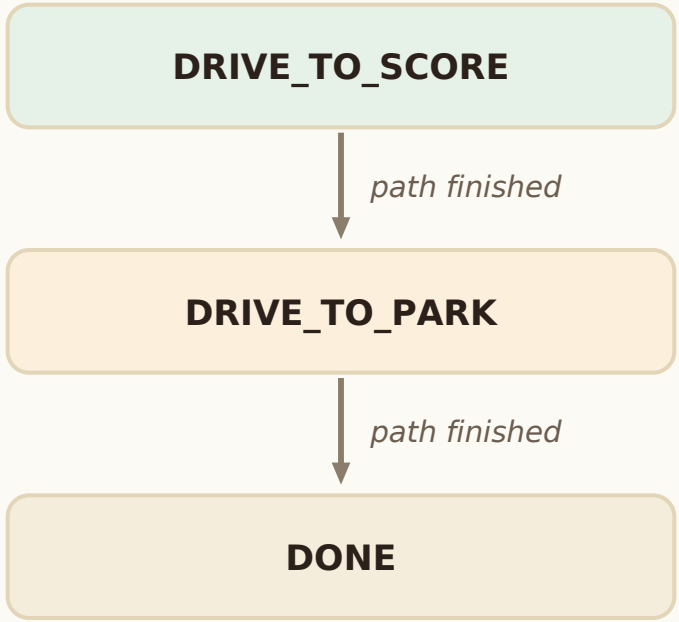
PathLab Test 1 is this program in miniature: one line from (24, 24) toward the End point, heading kept constant. A starts it, and the OpMode keeps calling update().

Read more: Pedro docs → Setting Up Your Autonomous, Creating a Pose, TeleOp Usage

STEP 3 · MANY PATHS

One step at a time: a state machine.

```
enum Step { DRIVE_TO_SCORE, DRIVE_TO_PARK, DONE }
Step step = Step.DRIVE_TO_SCORE;
// start():
follower.follow(scorePath());
// loop():
follower.update();
switch (step) {
  case DRIVE_TO_SCORE:
    if (!follower.isBusy()) {
      follower.follow(parkPath());
      step = Step.DRIVE_TO_PARK;
    }
    break;
  case DRIVE_TO_PARK:
    if (!follower.isBusy()) step = Step.DONE;
    break;
  case DONE: break;
}
```



Each pass of `loop()` only **checks** and moves on. It never waits.

BACK TO THE DEMO

PathLab's OpMode is the same idea with these steps: READY, DRIVING_PATH, PAUSING_AT_ENDPOINT, RETURNING, STOPPED. Press A and it moves from READY to DRIVING_PATH.

Read more: Pedro docs → Autonomous Usage (poses, paths), Follow States (isBusy). The state machine is our team's pattern.

STEP 3 · MANY PATHS

Is the path finished?

Check	True when…	Use it
`!follower.isBusy()`	the path is done **and** the robot has stopped at the target	our default
`follower.atParametricEnd()`	the robot reached the end of the path; it need not have stopped	looser; try it if `isBusy()` is slow

The docs also show an even looser check for faster autos (low speed and `distanceToEndpoint()` under 4). See the Follow States page.

```
// the default check:
if (!follower.isBusy()) {
  // start the next path
}

// a looser check:
if (follower.atParametricEnd()) {
  // start the next path
}
```

Check the finish **every pass** of `loop()`. Never `sleep()` or `while` inside it: the follower would stop getting updates.

Not sure which to use? Print `follower.mode()` to see what the follower is doing.

Read more: Pedro docs → Follow States

STEP 4 · HEADING

Choose how the robot faces.

Add to the path	The robot…	In the demo
`.tangent()`	faces the way it travels (the default)	Test 3
`.linear(startPose, park)`	turns smoothly from the first heading to the last	Test 2
`.constant(startPose)`	keeps one heading all the way	Test 1
`.facingPoint(target)`	keeps pointing at one spot	Test 7

```
// go deeper: tangent for the first half, then linear
path = path.heading(
    Interpolator.piecewise()
        .until(0.5, Interpolator.tangent)
        .until(1, Interpolator.linear(startPose, park))
);
```

Give these a **Pose**, like `startPose`. A bare number is read as **radians**.

Read more: Pedro docs → Interpolation reference; PathLab Path Tests User Guide

STEP 5 · OTHER ALLIANCE

Write the Auto once. Mirror it for red.

```
PoseFactory blue = PoseFactory.degrees();
PoseFactory red = blue.mirrorX(70.75);

Pose blueStart = blue.of(30, 30, 0);
Pose redStart = red.of(30, 30, 0);
```

	x	y
blueStart	30	30
redStart	111.5	30

The mirror line is the middle of the field: **x = 70.75**.

Check, do not assume. The docs do not say what mirroring does to the **heading**. Print `follower.pose()` for the red start and look at the field in the Visualizer.

Only the factory changes. Your path code stays the same.

BACK TO THE DEMO

On the Dashboard you can change the PathLab points yourself. Try mirrored x values (141.5 minus x) and watch the robot go to the other side of the field.

Read more: Pedro docs → Pose Factory (mirrorX)

The same follower can drive with sticks.

```
DrivePowers powers = ManualDrive.fieldCentric(
    -gamepad1.left_stick_y,
    gamepad1.left_stick_x,
    gamepad1.right_stick_x,
    follower.pose().heading()
);

follower.manual(powers);
follower.update();
```

Same follower. Same `update()` every loop. TeleOp is not a different system.

Drive style	Code
Robot-centric (sticks relative to the robot)	`follower.manual(forward, lateral, turn)`
Field-centric (sticks relative to the field)	`ManualDrive.fieldCentric(…)` needs good localization
Hold when sticks are free	`driveOrHold(follower, …)`

Read more: Pedro docs → TeleOp Usage

STEP 6 · TELEOP

Hand the pose from Auto to TeleOp.

```
public class OpModeStorage {
    public static Pose autonomousEndPose
        = new Pose(0, 0, 0);
}
```

A shared place to keep the pose.

```
// in the Auto:
@Override
public void stop() {
    OpModeStorage.autonomousEndPose
        = follower.pose();
}
```

```
// in the TeleOp:
@Override
public void start() {
    follower.setPose(
        OpModeStorage.autonomousEndPose);
    follower.update();
}
```

Field-centric driving needs a correct pose. The handover gives TeleOp the pose where Auto ended, so no re-aiming by hand.

If the robot is moved between Auto and TeleOp, the stored pose is wrong. Set the pose again.

Read more: Pedro docs → TeleOp Usage (passing your position from Autonomous to TeleOp)

STEP 7 · CHECK YOUR WORK

Print what Pedro thinks. Compare with the floor.

```
// in loop(), after follower.update():
telemetry.addData("X", follower.pose().x());
telemetry.addData("Y", follower.pose().y());
telemetry.addData("Heading",
    Math.toDegrees(follower.pose().heading()));
telemetry.addData("Mode", follower.mode());
telemetry.update();
```

Heading comes back in **radians**. `Math.toDegrees` turns it into the degrees you type.

Question	Look at
Where am I?	`follower.pose()`
What is it doing?	`follower.mode()`: FOLLOW, HOLD, IDLE or MANUAL
How far along?	`completion()`, `remainingDistance()`
Why this wheel power?	`follower.debug().drivetrain()` (also `.algorithm()`, `.localizer()`)
Plan vs robot?	FTC Dashboard field view

BACK TO THE DEMO

The Dashboard in the demo showed the planned path next to where the robot really was. That gap is what the follower works to close.

Read more: Pedro docs → Follow States, Debugging

STEP 7 · FIX MISTAKES

When it goes wrong, check these first.

What you see	Check
Robot does not move	`follower.update()` in every `loop()`? Did you call `follow(path)`?
Path starts in the wrong place	`setPose(startPose)` must match where you physically put the robot.
Path never moves on to the next step	Print `follower.mode()`. Try `atParametricEnd()` instead of `!isBusy()`.
Everything freezes or jerks	A `while` or `sleep()` inside `loop()`.
Heading is off by a lot	Degrees in, **radians** out. `pose().heading()` is radians.
Wrong shape, strafes, drifts	Probably not code. Use the Tuning Checklist.

Read more: Pedro docs → Setting Up Your Autonomous, Creating a Pose, Follow States, TeleOp Usage; Deck A

RECAP

Twelve calls you now own.

`Constants.create(hardwareMap)`	build the follower
`follower.setPose(pose)`	say where the robot is
`follower.follow(path)`	start a path
`follower.update()`	run every loop
`follower.isBusy()`	still going?
`follower.pose()`	where am I? (radians)

`PoseFactory.degrees()`	makes Poses in degrees
`p.of(x, y, heading)`	one Pose
`p.mirrorX(70.75)`	flip for red
`line(a, b) / curve(a, ctrl, b)`	make a path
`.tangent() .linear() …`	how it faces
`follower.manual(…)`	drive with sticks

Read more: Pedro docs; PathLab Path Tests User Guide

CHECK YOURSELF

Six questions. Say the answers out loud.

1	Why does `follower.update()` go in `loop()` and not in `init()`?
2	Your Auto drives path 1, then path 2. What check tells you path 1 is done?
3	Why is `while (follower.isBusy()) {}` a bad idea in `loop()`?
4	You wrote `p.of(24, 24, 0)` but put the robot at (30, 30). What goes wrong?
5	Which heading rule keeps the robot pointing at the scoring target while it drives?
6	Your blue Auto works. What do you check after mirroring it for red?

Read more: Answers are in the instructor notes.